

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO 5: Construct interoperable web services by leveraging JAX-RPC, WSDL, SOAP, and XML Schema for seamless data exchange across distributed systems. (L4, L5)

Assessment Tool : Industry Problem Task

Weightage: 40%

QUESTIONS:

Total Marks: 30

1. A hospital management system requires a **SOAP-based web service** to manage patient information such as registration, appointment scheduling, and medical records. The system must ensure secure, structured, and interoperable communication between different hospital departments.

Design a **SOAP-based web service** for the hospital system. Explain the structure of SOAP messages (Envelope, Header, Body), define the operations involved (e.g., add patient, get records), and illustrate how data is exchanged between client and server. Justify the use of SOAP in this scenario considering reliability and security.

2. An enterprise application needs to expose its web services so that external systems can understand how to interact with them. The organization plans to use **WSDL (Web Services Description Language)** to describe the service.

Design the **WSDL structure** for a web service that provides operations such as retrieving and updating data. Explain the key components of WSDL (types, message, portType, binding, service), and illustrate how they define the service interface and communication details. Provide a clear structure and justify its importance in service integration.

3. A company is experiencing issues in its SOAP-based communication system where requests are not processed correctly, and responses are either delayed or incorrect.

Analyze the SOAP communication process and identify possible issues such as message formatting errors, incorrect namespaces, or endpoint problems. Propose a debugging approach to resolve these issues, including validation of SOAP messages and error handling techniques. Explain how the corrected system ensures reliable communication.

Code of Conduct:

I G . M O H A M M A D S H A R E E F / (1 9 2 3 1 1 2 8 8) (Name / Reg No)
certify that this submission is my original work and that I have adhered to the
guidelines specified for this assessment. I understand that any violation of academic
integrity rules will result in disciplinary action.

Signature of the student: G. MOHAMMAD SHAREEF

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding ; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

ANSWERS

1. SOAP-Based Hospital Management Web Service

A hospital management system is a good use case for a **SOAP (Simple Object Access Protocol)** web service because patient information is sensitive and must be exchanged in a structured, secure, and interoperable manner.

Architecture

The basic communication is:

Hospital Client/Department → SOAP Request → Web Service Server → SOAP Response → Client

For example, the registration department can send an AddPatient request, while the medical-records department can send a GetMedicalRecords request.

SOAP Message Structure

A SOAP message consists mainly of three parts:

```
<soap:Envelope
  xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:hs="http://hospital.example.com/services">

  <soap:Header>
    <hs:Authentication>
      <hs:Username>departmentUser</hs:Username>
      <hs:Token>ABC123</hs:Token>
    </hs:Authentication>
  </soap:Header>

  <soap:Body>
    <hs:AddPatient>
      <hs:Patient>
        <hs:PatientId>P1001</hs:PatientId>
        <hs:Name>John Smith</hs:Name>
        <hs:Age>45</hs:Age>
        <hs:Gender>Male</hs:Gender>
      </hs:Patient>
    </hs:AddPatient>
  </soap:Body>

</soap:Envelope>
```

- **Envelope:** The outermost element. It identifies the message as a SOAP message and contains the Header and Body.
- **Header:** Contains optional information such as authentication, authorization, transaction information, and security tokens.
- **Body:** Contains the actual request or response data.
- **Fault:** If an error occurs, a SOAP Fault can be returned inside the Body.

Main Operations

The service could provide operations such as:

Operation	Purpose
AddPatient	Register a new patient
GetPatient	Retrieve patient details
UpdatePatient	Modify patient information
ScheduleAppointment	Create an appointment
CancelAppointment	Cancel an appointment
GetMedicalRecords	Retrieve medical records
UpdateMedicalRecord	Add or modify medical information

For example, a client can request medical records:

```
<soap:Body>
  <hs:GetMedicalRecords>
    <hs:PatientId>P1001</hs:PatientId>
  </hs:GetMedicalRecords>
</soap:Body>
```

The server processes the request and returns:

```
<soap:Body>
  <hs:GetMedicalRecordsResponse>
    <hs:PatientId>P1001</hs:PatientId>
    <hs:Diagnosis>...</hs:Diagnosis>
    <hs:Treatment>...</hs:Treatment>
  </hs:GetMedicalRecordsResponse>
</soap:Body>
```

Why SOAP is Suitable

SOAP is appropriate because:

- **Security:** SOAP supports standards such as WS-Security for authentication, message integrity, and encryption.
- **Reliability:** SOAP can work with enterprise messaging and reliable-delivery mechanisms.
- **Interoperability:** Different hospital systems and programming languages can communicate using a standardized XML format.
- **Structured communication:** XML schemas provide strict data structures.
- **Error handling:** SOAP Faults provide a standardized mechanism for reporting errors.
- **Transaction support:** SOAP-based enterprise services can integrate with transaction and other WS-* standards.

Thus, SOAP provides a suitable foundation when the hospital requires **secure, reliable, and standardized communication between departments and external systems.**

2. WSDL Structure for an Enterprise Web Service

WSDL (Web Services Description Language) describes a web service so that clients can understand **what operations are available, what data they accept/return, and how and where to communicate with the service.**

A typical WSDL structure is:

```
<definitions
```

```

name="EnterpriseDataService"
targetNamespace="http://example.com/data"
xmlns:tns="http://example.com/data"
xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/">

<types>
  <xsd:schema targetNamespace="http://example.com/data">
    <xsd:element name="GetDataRequest">
      <xsd:complexType>
        <xsd:sequence>
          <xsd:element name="id" type="xsd:int"/>
        </xsd:sequence>
      </xsd:complexType>
    </xsd:element>

    <xsd:element name="UpdateDataRequest">
      <xsd:complexType>
        <xsd:sequence>
          <xsd:element name="id" type="xsd:int"/>
          <xsd:element name="value" type="xsd:string"/>
        </xsd:sequence>
      </xsd:complexType>
    </xsd:element>
  </xsd:schema>
</types>

<message name="GetDataInput">
  <part name="parameters" element="tns:GetDataRequest"/>
</message>

<message name="GetDataOutput">
  <part name="parameters" element="tns:GetDataResponse"/>
</message>

<message name="UpdateDataInput">
  <part name="parameters" element="tns:UpdateDataRequest"/>
</message>

<message name="UpdateDataOutput">
  <part name="parameters" element="tns:UpdateDataResponse"/>
</message>

<portType name="DataPortType">
  <operation name="GetData">
    <input message="tns:GetDataInput"/>

```

```

        <output message="tns:GetDataOutput"/>
    </operation>

    <operation name="UpdateData">
        <input message="tns:UpdateDataInput"/>
        <output message="tns:UpdateDataOutput"/>
    </operation>
</portType>

<binding name="DataSoapBinding" type="tns:DataPortType">
    <soap:binding
        transport="http://schemas.xmlsoap.org/soap/http"
        style="document"/>

    <operation name="GetData">
        <soap:operation soapAction="GetData"/>
    </operation>

    <operation name="UpdateData">
        <soap:operation soapAction="UpdateData"/>
    </operation>
</binding>

<service name="EnterpriseDataService">
    <port name="DataSoapPort" binding="tns:DataSoapBinding">
        <soap:address location="https://example.com/DataService"/>
    </port>
</service>

</definitions>

```

Key Components

- **Types:** Defines the data types used by the service, usually using XML Schema (XSD). For example, it can define an integer id and a string value.
- **Message:** Defines the data exchanged between client and service. Messages normally represent request and response information.
- **PortType:** Defines the **abstract interface** of the service. It lists operations such as GetData and UpdateData and specifies their input/output messages.
- **Binding:** Defines how the abstract operations are communicated. It can specify SOAP, HTTP transport, document style, SOAP actions, and related communication details.
- **Service:** Provides the actual service endpoint. It connects a binding to a physical URL where the service can be accessed.
- **Port:** Specifies a particular endpoint and its binding.

Relationship Between Components

The relationship can be summarized as:

Types → Messages → PortType → Binding → Service/Port

For example:

GetData

- uses GetDataInput and GetDataOutput
- defined in DataPortType
- implemented using DataSoapBinding
- exposed through <https://example.com/DataService>

Importance of WSDL

WSDL is important because it provides a **machine-readable contract** between service providers and consumers. A client can inspect the WSDL to automatically determine:

- Which operations are available
- What parameters are required
- What responses are returned
- Which communication protocol is used
- Where the service is located

This reduces ambiguity and makes integration between independently developed enterprise systems easier. Tools can also generate client-side code from a WSDL document, reducing development effort.

3. Debugging Problems in SOAP Communication

When SOAP requests are not processed correctly or responses are delayed/incorrect, debugging should examine the communication from the client through the server and back to the client.

SOAP Communication Process

The normal process is:

Client creates SOAP request

↓

Request sent to endpoint

↓

Server receives and parses XML

↓

Server validates SOAP structure and namespaces

↓

Service operation is identified

↓

Business logic executes

↓

SOAP response is generated

↓

Response returned to client

An error at any stage can cause communication failure.

Possible Problems

1. Incorrect SOAP message formatting

The XML may contain:

- Missing closing tags

- Invalid XML
- Incorrect SOAP Envelope structure
- Incorrect element names
- Unexpected elements
- Missing required parameters

For example, a malformed request might fail before the application logic is reached.

Solution: Validate the XML against the appropriate XML Schema/XSD and inspect the complete SOAP request.

2. Incorrect namespaces

Namespaces are particularly important in SOAP. A request can look structurally correct but still fail if it uses the wrong namespace.

For example:

```
<hs:GetPatient
  xmlns:hs="http://wrong.example.com">
```

while the server expects:

```
<hs:GetPatient
  xmlns:hs="http://hospital.example.com/services">
```

The server may therefore fail to recognize the requested operation.

Solution: Compare the namespaces in the request with those specified by the WSDL and XSD.

3. Wrong endpoint URL

The client may be sending the request to an incorrect or outdated service URL.

Possible causes include:

- Incorrect hostname
- Wrong port
- Incorrect service path
- Service moved to a different server
- DNS/network problems

Solution: Verify the endpoint in the WSDL and test connectivity independently.

4. Incorrect SOAPAction or operation

For SOAP 1.1, the HTTP request may contain a SOAPAction value identifying the operation. If this does not correspond to what the server expects, the request may not be dispatched correctly.

Solution: Compare the SOAPAction and operation name with the WSDL binding.

5. SOAP version mismatch

SOAP 1.1 and SOAP 1.2 have differences in namespaces and HTTP content types.

For example, the client and server may use different SOAP versions.

Solution: Confirm that both sides use the same SOAP version and compatible binding.

6. Authentication and security failures

If authentication tokens, certificates, or WS-Security headers are missing or invalid, the server may reject the request.

Solution: Check security headers, credentials, certificates, timestamps, and server security logs without exposing sensitive credentials.

7. Server-side processing errors

The SOAP message may be valid, but the application can fail because of:

- Database errors
- Invalid business data
- Null values
- Application exceptions
- Incorrect service configuration

These should result in an appropriate SOAP Fault rather than an unexplained or malformed response.

Recommended Debugging Approach

A systematic debugging process is:

Capture the complete SOAP request and response.

Use a SOAP testing/debugging tool or server-side logging.

Validate the XML.

Check whether the message is well-formed XML.

Validate against the WSDL/XSD.

Confirm that element names, data types, required fields, and namespaces are correct.

Check the SOAP Envelope.

Verify the Envelope, Header, and Body structure.

Check namespaces.

Compare every relevant namespace with the WSDL.

Check endpoint and HTTP settings.

Verify URL, HTTP method, content type, SOAPAction, TLS configuration, and network connectivity.

Check authentication/security.

Verify that required security headers and credentials are valid.

Check server logs.

Determine whether the request reached the service and whether an application or database error occurred.

Test with a known-good request.

Send a request generated directly from the WSDL or a working SOAP client and compare it with the failing request.

Check response timing.

If responses are delayed, investigate database queries, network latency, server load, timeouts, and downstream services.

SOAP Fault Handling

Instead of returning an unclear response, the server should provide a standardized SOAP Fault:

```
<soap:Fault>
  <faultcode>soap:Client</faultcode>
  <faultstring>Invalid Patient ID</faultstring>
  <detail>
    <ErrorCode>PATIENT_001</ErrorCode>
  </detail>
</soap:Fault>
```

The client should distinguish between:

- **Client errors:** malformed requests, invalid parameters, authentication problems.
- **Server errors:** internal application failures, database failures, unavailable dependencies.
- **Communication errors:** timeouts, DNS failures, connection failures.

For transient failures, appropriate timeout and retry policies can be used, while avoiding uncontrolled repeated requests that could create duplicate operations.

Result

After correction, the system should have a consistent chain:

Valid SOAP request → Correct namespace → Correct endpoint → Successful authentication → Correct operation → Server processing → Valid SOAP response/Fault

This approach improves reliability because problems are detected at the appropriate layer, errors are communicated using standardized SOAP Faults, and both client and server can diagnose failures without guessing where the communication broke.